

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Base de Datos II

Funciones para Manipulación de Números

Autor: Angel Simón

Índice

1. Introducción	2
2. Base de Datos de Ejemplo	2
2.1. Descripción de los Campos	3
3. Operaciones Matemáticas Básicas	3
4. Funciones Integradas para Números	4
4.1. ABS() - Valor Absoluto	4
4.2. SIGN() - Signo del Número	4
4.3. CEILING() - Redondeo hacia Arriba	5
4.4. FLOOR() - Redondeo hacia Abajo	5
4.5. ROUND() - Redondeo con Precisión	6
4.6. CAST() - Conversión de Tipos	6
5. Datos de Prueba	7

1. Introducción

En el contexto de una base de datos, es común almacenar valores numéricos que posteriormente requieren transformaciones u operaciones matemáticas. SQL proporciona un conjunto de operadores y funciones integradas para realizar estas manipulaciones de manera eficiente.

Este documento tiene como objetivo ejemplificar los casos más habituales de manipulación de números en SQL Server, presentando las funciones y operadores más utilizados en el desarrollo de consultas y reportes.

2. Base de Datos de Ejemplo

Para proporcionar un contexto uniforme y comprensible, se utilizará una base de datos simplificada que simula cuentas bancarias. El siguiente script permite crear la estructura necesaria en SQL Server:

```
Create Database EjemploFuncionesNumericas
Go
Use EjemploFuncionesNumericas
Go
Create Table Cuentas(
    ID_Cuenta bigint not null primary key identity (1, 1),
    DNI varchar(10) not null,
    Saldo decimal(18,2),
    Limite_Descubierto decimal(10,2),
    Interes_Mensual decimal(5,2),
    Cuota_Mensual decimal(10,2)
)
```

Código 1. Script de creación de la base de datos

Simplificación del Modelo

Este modelo representa una simplificación con fines didácticos. En un sistema real, el saldo debería calcularse a partir de transacciones almacenadas en tablas separadas. Asimismo, los valores de interés y cuota mensual deberían contar con tablas de histórico que permitan rastrear sus cambios a lo largo del tiempo.

2.1. Descripción de los Campos

La **Tabla 1** detalla el propósito de cada campo en la tabla Cuentas, que representa de manera simplificada cuentas bancarias con sus características principales.

Campo	Descripción
ID_Cuenta	Clave primaria y código único de la cuenta bancaria.
DNI	Identificador del titular de la cuenta. En un modelo completo, sería ID_Cliente con referencia a una tabla Clientes.
Saldo	Saldo actual de la cuenta bancaria.
Limite_Descubierto	Límite de descubierto permitido. Representa el monto máximo que la cuenta puede adeudar. Podría implementarse como restricción CHECK.
Interes_Mensual	Porcentaje de interés mensual que otorga el banco por mantener fondos en la cuenta.
Cuota_Mensual	Costo mensual de mantenimiento de la cuenta bancaria.

Tabla 1. Descripción de campos de la tabla Cuentas

3. Operaciones Matemáticas Básicas

SQL Server proporciona operadores estándar para realizar cálculos aritméticos con columnas numéricas. La **Tabla 2** resume los operadores matemáticos más utilizados.

Operador	Descripción	Ejemplo
+	Suma	Saldo + 5000
-	Resta	Saldo - Cuota_Mensual
	Multiplicación	Saldo * 0.15
/	División	Saldo / 2
%	Módulo (resto)	Saldo % 2

Tabla 2. Operadores matemáticos en SQL

Orden de Precedencia

Los operadores matemáticos siguen las reglas de precedencia estándar: multiplicación y división se evalúan antes que suma y resta. Utiliza paréntesis para especificar un orden diferente de evaluación.

4. Funciones Integradas para Números

SQL Server incluye funciones específicas para la manipulación de valores numéricos. Estas funciones son fundamentales para realizar cálculos, redondeos y evaluaciones de características numéricas en las consultas.

4.1. ABS() – Valor Absoluto

Propósito: Devuelve el valor absoluto de un número, eliminando su signo.

Sintaxis: ABS(expresion_numerica)

```
SELECT ID_Cuenta, Saldo, ABS(Saldo) AS Saldo_Absoluto
FROM Cuentas;
```

Código 2. Ejemplo de uso de ABS()

Aplicación práctica: Esta función resulta útil para calcular totales o promedios sin considerar si los valores son positivos o negativos, por ejemplo, al determinar el monto total en circulación independientemente de si representa crédito o débito.

Ejemplo de resultado: Si el saldo es -200.50, ABS(Saldo) devolverá 200.50.

4.2. SIGN() – Signo del Número

Propósito: Retorna el signo de un número como valor entero.

Sintaxis: SIGN(expresion_numerica)

Valores de retorno:

- 1 si el número es positivo
- 0 si el número es cero
- -1 si el número es negativo

```
SELECT ID_Cuenta, Saldo, SIGN(Saldo) AS Signo_Saldo
FROM Cuentas;
```

Código 3. Ejemplo de uso de SIGN()

Aplicación práctica: Permite identificar rápidamente cuentas con saldo deudor (negativo) o acreedor (positivo), facilitando la clasificación y generación de reportes segmentados.

Ejemplo de resultado: Si el saldo es 300.00, el resultado será 1; si es -50.00, será -1.

4.3. CEILING() – Redondeo hacia Arriba

Propósito: Redondea un número hacia arriba al entero inmediato superior.

Sintaxis: CEILING(expresion_numerica)

```
SELECT ID_Cuenta, Interes_Mensual,
       CEILING(Interes_Mensual) AS Interes_Redondeado_Arriba
FROM Cuentas;
```

Código 4. Ejemplo de uso de CEILING()

Aplicación práctica: Útil para realizar cálculos conservadores en reportes financieros o cuando se requiere garantizar un valor mínimo en estimaciones.

Ejemplo de resultado: Si Interes_Mensual es 3.25, CEILING(Interes_Mensual) devuelve 4.

4.4. FLOOR() – Redondeo hacia Abajo

Propósito: Redondea un número hacia abajo al entero inmediato inferior.

Sintaxis: FLOOR(expresion_numerica)

```
SELECT ID_Cuenta, Interes_Mensual,
       FLOOR(Interes_Mensual) AS Interes_Redondeado_Abajo
FROM Cuentas;
```

Código 5. Ejemplo de uso de FLOOR()

Aplicación práctica: Ideal para cálculos más conservadores al otorgar beneficios o cuando se requiere un valor máximo en descuentos.

Ejemplo de resultado: Si Interes_Mensual es 3.75, FLOOR(Interes_Mensual) devuelve 3.

4.5. ROUND() – Redondeo con Precisión

Propósito: Redondea un número a la cantidad especificada de decimales.

Sintaxis: ROUND(expresion_numerica, longitud)

```
SELECT ID_Cuenta, Saldo, ROUND(Saldo, 1) AS Saldo_Redondeado
FROM Cuentas;
```

Código 6. Ejemplo de uso de ROUND()

Aplicación práctica: Permite ajustar la precisión de valores numéricos para reportes simplificados o cálculos que no requieren alta exactitud decimal.

Ejemplo de resultado: Si el saldo es 123.456, ROUND(Saldo, 1) devuelve 123.5.

Comportamiento del Redondeo

ROUND() sigue la regla de redondeo estándar: si el dígito siguiente al especificado es 5 o mayor, redondea hacia arriba; caso contrario, hacia abajo.

4.6. CAST() – Conversión de Tipos

Propósito: Convierte un valor de un tipo de datos a otro tipo especificado.

Sintaxis: CAST(expresion AS tipo_datos)

```
SELECT ID_Cuenta, CAST(Saldo AS Integer) AS Saldo_Sin_Centavos
FROM Cuentas;
```

Código 7. Ejemplo de uso de CAST()

Aplicación práctica: En este contexto, CAST() transforma el saldo de tipo DECIMAL(18,2) a un número entero (INT), truncando los decimales. Resulta útil para reportes donde no se requiere precisión decimal o para simplificar cálculos.

Ejemplo de resultado:

- Si el saldo es 1234.56, el resultado será 1234
- Si el saldo es -567.89, el resultado será -567

Versatilidad de CAST()

Aunque en este apunte se presenta CAST() en el contexto de valores numéricos, esta función no es exclusiva de números. Puede emplearse para convertir entre diversos tipos de datos siempre que la conversión sea lógicamente válida.

5. Datos de Prueba

A continuación se proporciona un script con datos de ejemplo para poblar la tabla Cuentas y poder ejecutar las consultas presentadas en este documento:

```
Use EjemploFuncionesNumericas;

INSERT INTO Cuentas (DNI, Saldo, Limite_Descubierto,
                    Interes_Mensual, Cuota_Mensual)
VALUES
    ('123456789', 1500.75, 500.00, 1.25, 10.00),
    ('987654321', -250.50, 1000.00, 2.00, 15.00),
    ('562348791', 0.00, 200.00, 1.50, 12.50),
    ('741258963', 3000.00, 750.00, 1.75, 20.00),
    ('951357468', -100.25, 600.00, 1.00, 8.00),
    ('354789621', 0.00, 300.00, 2.50, 10.50),
    ('246813579', -500.00, 800.00, 1.25, 15.75),
    ('159487263', 200.00, 150.00, 1.00, 5.00),
    ('753961842', 5000.50, 1000.00, 2.25, 25.00),
    ('369258147', -300.00, 1200.00, 1.75, 18.50);
```

Código 8. Script de inserción de datos de prueba

Variedad de Casos

Los datos de prueba incluyen deliberadamente cuentas con saldos positivos, negativos y en cero, lo que permite verificar el comportamiento de todas las funciones presentadas en diferentes escenarios.